

Bit Manipulation Unit (BMU)

Functional Specification — Verification-Ready Edition

Document Version: 1.2 (Verification-Consolidated)

Base Document: BMU Specification v1.1

Consolidated by: Verification Team

Document Control

This document consolidates the original BMU Specification (v1.1) with corrections and clarifications obtained from the design team during verification kickoff. It is intended as the single reference for both design and verification going forward.

Change	Description	Source
Table error-value corrections	OR, XOR, SRL, SRA, ROR, BINV: 'other fields $\neq 0$ ' invalid rows corrected from error=0 to error=1.	Design team clarification
CTZ(0) correction	All-zero input now correctly specified to return 32, not 0, per RISC-V convention.	Design team clarification
Timing & reset behavior added	Section 3 added — latency, valid_in gating, and reset behavior were previously undocumented.	Design team clarification
GREV undefined-encoding behavior	Adopted assumption — see Section 7.	Verification team assumption, pending confirmation
CSR conflict condition scope	Adopted assumption — see Section 7.	Verification team assumption, pending confirmation
Known documentation gaps	Section 8 added — fields observed in the control struct with no behavior specification.	Verification team, RTL cross-check

Important: Sections 7 and 8 contain items NOT yet confirmed by the design/spec owner. They are clearly marked and must be resolved before being treated as final.

Document Control	2
1. Introduction & Scope	6
1.1 Scope of this document	6
2. Interface Definition.....	6
3. Timing, Reset & Test-Mode Behavior	6
3.1 Latency	6
3.2 valid_in = 0 Behavior.....	6
3.3 Reset Behavior.....	7
3.4 scan_mode	7
4. Guard-Condition Philosophy	7
5. Global Error Conditions.....	7
6. Operation Behavior Specifications	7
6.1 Logic Operations	7
6.1.1 OR (ap.lor)	7
Operation Control Signals	7
Execution Guard Conditions.....	7
Behavior Specification	8
Worked Example	8
6.1.2 XOR (ap.lxor).....	8
Operation Control Signals	8
Execution Guard Conditions.....	8
Behavior Specification	8
Worked Example	8
6.2 Shift & Bit-Index Operations.....	8
6.2.1 SRL — Shift Right Logical (ap.srl)	8
Operation Control Signals	8
Execution Guard Conditions.....	8
Behavior Specification	8
Worked Example	9
6.2.2 SRA — Shift Right Arithmetic (ap.sra)	9
Operation Control Signals	9
Execution Guard Conditions.....	9
Behavior Specification	9
Worked Example	9
6.2.3 ROR — Rotate Right (ap.ror)	9
Operation Control Signals	9
Execution Guard Conditions.....	9
Behavior Specification	9

Worked Example	9
6.2.4 BINV — Bit Invert (ap.binv)	9
Operation Control Signals	10
Execution Guard Conditions	10
Behavior Specification	10
Worked Example	10
6.3 Address-Generation (Zba)	10
6.3.1 SH2ADD — Shift Left by 2 and Add (ap.sh2add)	10
Operation Control Signals	10
Execution Guard Conditions	10
Behavior Specification	10
Worked Example	10
6.4 Arithmetic & Comparison	10
6.4.1 SUB — Subtraction (ap.sub)	10
Operation Control Signals	11
Execution Guard Conditions	11
Behavior Specification	11
Worked Example	11
6.4.2 SLT / SLTU — Set on Less Than (ap.slt, co-requisite ap.sub)	11
Operation Control Signals	11
Execution Guard Conditions	11
Behavior Specification	11
Worked Example	11
6.5 Count & Population Operations	11
6.5.1 CTZ — Count Trailing Zero Bits (ap.ctz)	11
Operation Control Signals	12
Execution Guard Conditions	12
Behavior Specification	12
Worked Example	12
6.5.2 CPOP — Population Count (ap.cpop)	12
Operation Control Signals	12
Execution Guard Conditions	12
Behavior Specification	12
Worked Example	12
6.6 Sign Extension	12
6.6.1 SEXT.B — Sign-Extend Byte (ap.sixt_b)	12
Operation Control Signals	12
Execution Guard Conditions	12

Behavior Specification	13
Worked Example	13
6.7 Min/Max.....	13
6.7.1 MAX (ap.max, co-requisite ap.sub).....	13
Operation Control Signals	13
Execution Guard Conditions.....	13
Behavior Specification	13
Worked Example	13
6.8 Data Packing & Permutation	13
6.8.1 PACK (ap.pack).....	13
Operation Control Signals	13
Execution Guard Conditions.....	13
Behavior Specification	13
Worked Example	14
6.8.2 GREV — Byte-Reverse subset (ap.grev)	14
Operation Control Signals	14
Execution Guard Conditions.....	14
Behavior Specification	14
Worked Example	14
6.9 CSR Operations	14
6.9.1 CSR Bypass Read (csr_ren_in, ap = 0)	14
Operation Control Signals	14
Execution Guard Conditions.....	14
Behavior Specification	14
Worked Example	14
6.9.2 CSR Write Data (ap.csr_write)	15
Operation Control Signals	15
Execution Guard Conditions.....	15
Behavior Specification	15
7. Adopted Assumptions Pending Design-Team Confirmation.....	15
7.1 GREV undefined-encoding behavior.....	15
7.2 CSR-conflict condition scope (AND vs. OR)	15
8. Known Documentation Gaps (Out of Current Scope)	15
Appendix A: Revision Summary vs. v1.1.....	16

1. Introduction & Scope

The Bit Manipulation Unit (BMU) is a synthesizable, combinational RTL block that implements a defined subset of the RISC-V Bit Manipulation extension (Zbb, Zbs, Zbp, Zba). It computes a single result per cycle from two 32-bit operands, based on which control field in the decoded instruction struct (ap) is asserted.

This specification defines, for every supported operation: its enabling control field(s), the conditions under which it is valid, its result expression, and its error behavior. It also defines the block's timing, reset, and test-mode behavior.

1.1 Scope of this document

This document covers exactly the operations for which the original specification provided a full behavior table, plus the timing/reset/scan behavior obtained separately. Section 8 explicitly lists fields that exist in the control struct but are not covered here, so scope is never ambiguous by omission.

2. Interface Definition

Signal	Width	Dir	Description
clk	1	in	Clock.
rst_l	1	in	Active-low synchronous reset. Clears result_ff and error to 0.
scan_mode	1	in	DFT scan-test control. Not used in functional operation; tie to 0 for functional verification.
valid_in	1	in	Marks a_in / b_in / ap as valid for this cycle. Enables result_ff to capture the new result.
a_in	32	in	First operand.
b_in	32	in	Second operand (also carries shift/bit-index amounts in its low bits for shift/bit-index ops).
ap	struct	in	Decoded per-operation enable fields (one bit per supported operation).
csr_ren_in	1	in	CSR read-enable (bypass-read request).
csr_rddata_in	32	in	CSR read data, valid when csr_ren_in = 1.
result_ff	32	out	Registered result. Updated on the clock edge when valid_in = 1; holds otherwise.
error	1	out	Combinational error flag — reflects current inputs every cycle, independent of valid_in.

3. Timing, Reset & Test-Mode Behavior

Note: This entire section was undocumented in the original specification. Content is sourced from an informal design-team clarification and has not yet been captured in a formally reviewed spec revision. Treat as high-confidence but not yet signed off.

3.1 Latency

The BMU is single-cycle latency: all operations are computed combinatorially from a_in, b_in and ap. The combinational result is captured into result_ff on the clock edge. If valid_in is asserted with a given input set in cycle N, the corresponding result is present in result_ff in cycle N+1.

3.2 valid_in = 0 Behavior

- result_ff holds its previous value — the register's clock-enable is valid_in.
- error does NOT hold. It is purely combinational and reflects whatever is on the inputs every cycle, regardless of valid_in.

3.3 Reset Behavior

On rst_l = 0 (synchronous): result_ff clears to 0, and error clears to 0, overriding all other input conditions.

3.4 scan_mode

Used only for scan-chain DFT insertion. Has no functional role. Must be held at 0 for all functional verification.

4. Guard-Condition Philosophy

Every operation below defines an Execution Guard Condition: exactly one primary enable field in ap may be asserted, all other ap.* fields must be 0, and csr_ren_in must be 0. Violating any guard condition invalidates the operation: result is forced to 0 and error is asserted (error = 1), unless otherwise noted for a specific operation (see SLT/MAX/SH2ADD, which legitimately require a second co-requisite field).

5. Global Error Conditions

In addition to the per-operation guard conditions, three conditions apply chip-wide:

- Simultaneous activation of a CSR read request (csr_ren_in = 1) and any bit-manipulation control field.
- Use of ADD/SUB while a Zba (address-generation) operation is simultaneously selected without the correct co-requisite fields.
- Use of a SHxADD control field without the required ap.zba mode bit asserted.

6. Operation Behavior Specifications

6.1 Logic Operations

Controlled by ap.lor / ap.lxor, modified by ap.zbb (bitwise-invert-b variant). Valid only when csr_ren_in = 0 and no other ap.* field is set.

6.1.1 OR (ap.lor)

Bitwise OR of a_in and b_in, or OR with inverted b_in when ap.zbb = 1.

Operation Control Signals

- Primary Enable: ap.lor = 1
- Mode Select: ap.zbb (0 = OR, 1 = OR with ~b_in)

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	ap.zbb	Result	Error
Valid OR	0	result = a_in b_in	0
Valid OR (inverted)	1	result = a_in ~b_in	0
Invalid — other ap.* field ≠ 0	—	result = 0	1
Invalid — csr_ren_in = 1 (with ap.lor = 1)	—	result = 0	1

Worked Example

```
a_in=0xF0F0_F0F0; b_in=0xF0F0_F0F0; ap.lor=1; ap.zbb=0; // result = 0xFFFF_FFFF, error = 0
```

6.1.2 XOR (ap.lxor)

Bitwise XOR of a_in and b_in, or XOR with inverted b_in (XNOR-like) when ap.zbb = 1.

Operation Control Signals

- Primary Enable: ap.lxor = 1
- Mode Select: ap.zbb (0 = XOR, 1 = XOR with ~b_in)

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	ap.zbb	Result	Error
Valid XOR	0	result = a_in ^ b_in	0
Valid XOR (inverted)	1	result = a_in ^ ~b_in	0
Invalid — other ap.* field ≠ 0	—	result = 0	1
Invalid — csr_ren_in = 1 (with ap.lxor = 1)	—	result = 0	1

Worked Example

```
a_in=0xAAAA_AAAA; b_in=0x5555_5555; ap.lxor=1; ap.zbb=1; // result = a_in ^ ~b_in = 0x0000_0000, error = 0
```

6.2 Shift & Bit-Index Operations

One shift/bit-index control field may be active at a time. Shift/index amount is b_in[4:0].

6.2.1 SRL — Shift Right Logical (ap.srl)

Logical (zero-fill) right shift of a_in by b_in[4:0].

Operation Control Signals

- Primary Enable: ap.srl = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid SRL	result = a_in >> b_in[4:0]	0
Invalid — other ap.* field ≠ 0	result = 0	1
Invalid — csr_ren_in = 1 (with ap.srl = 1)	result = 0	1

Worked Example

```
a_in=0xF000_0000; b_in=4; ap.srl=1; // result = 0x0F00_0000, error = 0
```

6.2.2 SRA — Shift Right Arithmetic (ap.sra)

Sign-preserving right shift of a_in by b_in[4:0]; fills with the sign bit.

Operation Control Signals

- Primary Enable: ap.sra = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid SRA	result = \$signed(a_in) >>> b_in[4:0]	0
Invalid — other ap.* field ≠ 0	result = 0	1
Invalid — csr_ren_in = 1 (with ap.sra = 1)	result = 0	1

Worked Example

```
a_in=0xF000_0000; b_in=4; ap.sra=1; // result = 0xFF00_0000 (sign-extended), error = 0
```

6.2.3 ROR — Rotate Right (ap.ror)

Rotates a_in right by b_in[4:0]; bits shifted out the LSB re-enter at the MSB.

Operation Control Signals

- Primary Enable: ap.ror = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid ROR	result = (a_in >> b_in[4:0]) (a_in << (32-b_in[4:0]))	0
Invalid — other ap.* field ≠ 0	result = 0	1
Invalid — csr_ren_in = 1 (with ap.ror = 1)	result = 0	1

Worked Example

```
a_in=0x89ABCDE; b_in=4; ap.ror=1; // result = 0xF89ABCDE, error = 0
```

6.2.4 BINV — Bit Invert (ap.binv)

Inverts a single bit of `a_in` at position `b_in[4:0]`; all other bits unchanged.

Operation Control Signals

- Primary Enable: `ap.binv = 1`

Execution Guard Conditions

- All other `ap.*` fields must be 0
- `csr_ren_in` must be 0

Behavior Specification

Condition	Result	Error
Valid BINV	$\text{result} = \text{a_in} \wedge (32'b1 \ll \text{b_in}[4:0])$	0
Invalid — other <code>ap.*</code> field $\neq 0$	$\text{result} = 0$	1
Invalid — <code>csr_ren_in = 1</code> (with <code>ap.binv = 1</code>)	$\text{result} = 0$	1

Worked Example

```
a_in=0xFFFF_FFFF; b_in=2; ap.binv=1; // bit 2 inverted (1->0): result = 0xFFFF_FFFB,
error = 0
```

6.3 Address-Generation (Zba)

6.3.1 SH2ADD — Shift Left by 2 and Add (`ap.sh2add`)

Computes $(\text{a_in} \ll 2) + \text{b_in}$. Requires the Zba mode bit. Used for scaled-index address generation.

Operation Control Signals

- Primary Enable: `ap.sh2add = 1`
- Required Mode: `ap.zba = 1`

Execution Guard Conditions

- `ap.zba` must be 1
- All other `ap.*` fields must be 0 except `ap.sh2add` and `ap.zba`
- `csr_ren_in` must be 0

Behavior Specification

Condition	Result	Error
Valid SH2ADD	$\text{result} = (\text{a_in} \ll 2) + \text{b_in}$	0
Invalid — <code>ap.zba</code> $\neq 1$	$\text{result} = 0$	1
Invalid — other <code>ap.*</code> field $\neq 0$ and <code>csr_ren_in = 1</code>	$\text{result} = 0$	1

Worked Example

```
a_in=4; b_in=7; ap.sh2add=1; ap.zba=1; // result = (4<<2)+7 = 23, error = 0
```

6.4 Arithmetic & Comparison

SUB, SLT and MAX all share the adder/subtractor datapath, so each legitimately requires a co-requisite second field (see each subsection) — this is a real hardware sharing pattern, not a guard-condition violation.

6.4.1 SUB — Subtraction (`ap.sub`)

Computes $a_in - b_in$. Must not be asserted simultaneously with a Zba operation unless that operation's own guard requires it (see Section 5).

Operation Control Signals

- Primary Enable: $ap.sub = 1$

Execution Guard Conditions

- All other $ap.*$ fields must be 0
- csr_ren_in must be 0
- $ap.zba$ must be 0 (plain SUB is not a Zba operation)

Behavior Specification

Condition	Result	Error
Valid SUB	$result = a_in - b_in$	0
Invalid — $ap.zba = 1$ with plain SUB	$result = 0$	1
Invalid — other $ap.*$ field $\neq 0$ and $csr_ren_in = 1$	$result = 0$	1

Worked Example

```
a_in=20; b_in=7; ap.sub=1; // result = 13, error = 0
```

6.4.2 SLT / SLTU — Set on Less Than ($ap.slt$, co-requisite $ap.sub$)

Compares a_in and b_in ; $result = 1$ if $a_in < b_in$, else 0. Signed unless $ap.unsign = 1$. Implemented via the shared subtractor, so $ap.sub = 1$ is a required co-requisite alongside $ap.slt = 1$.

Operation Control Signals

- Primary Enable: $ap.slt = 1$ together with $ap.sub = 1$
- Signed/Unsigned Select: $ap.unsign$ (0 = signed / SLT, 1 = unsigned / SLTU)

Execution Guard Conditions

- All other $ap.*$ fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	$ap.unsign$	Result	Error
Valid SLT	0	$result = (\$signed(a_in) < \$signed(b_in)) ? 1 : 0$	0
Valid SLTU	1	$result = (a_in < b_in) ? 1 : 0$ (unsigned)	0
Invalid — other $ap.*$ field $\neq 0$ and $csr_ren_in = 1$	—	$result = 0$	1

Worked Example

```
a_in=-2 (0xFFFFFFFEE); b_in=1; ap.slt=1; ap.sub=1; ap.unsign=0; // result = 1 (-2 < 1), error = 0
```

6.5 Count & Population Operations

6.5.1 CTZ — Count Trailing Zero Bits ($ap.ctz$)

Counts consecutive zero bits from the LSB until the first 1 bit.

Operation Control Signals

- Primary Enable: ap.ctz = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid, a_in ≠ 0	result = number of trailing zero bits in a_in	0
a_in = 0 (all zeros)	result = 32 (corrected — was 0 in v1.1)	0
Invalid — other ap.* field ≠ 0 and csr_ren_in = 1	result = 0	1

Worked Example

```
a_in=0x0000_0100; ap.ctz=1; // result = 8, error = 0
```

6.5.2 CPOP — Population Count (ap.cpop)

Counts the number of bits set to 1 in a_in.

Operation Control Signals

- Primary Enable: ap.cpop = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid CPOP	result = number of bits set to 1 in a_in	0
Invalid — other ap.* field ≠ 0 and csr_ren_in = 1	result = 0	1

Worked Example

```
a_in=0xF0F0_F00F; ap.cpop=1; // result = 16, error = 0
```

6.6 Sign Extension

6.6.1 SEXT.B — Sign-Extend Byte (ap.sixt_b)

Sign-extends bits [7:0] of a_in to the full 32-bit width, replicating bit 7 into bits [31:8].

Operation Control Signals

- Primary Enable: ap.sixt_b = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid SEXT.B	result = {{24{a_in[7]}}, a_in[7:0]}	0
Invalid — other ap.* field ≠ 0 and csr_ren_in = 1	result = 0	1

Worked Example

```
a_in=0x0000_007F; ap.sixt_b=1; // result = 0x0000_007F (positive, unchanged), error = 0
```

6.7 Min/Max

6.7.1 MAX (ap.max, co-requisite ap.sub)

Returns the larger of two signed operands. Implemented via the shared subtractor, so ap.sub = 1 is a required co-requisite alongside ap.max = 1.

Operation Control Signals

- Primary Enable: ap.max = 1 together with ap.sub = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid MAX	result = (\$signed(a_in) > \$signed(b_in)) ? a_in : b_in	0
Invalid — other ap.* field ≠ 0 and csr_ren_in = 1	result = 0	1

Worked Example

```
a_in=10; b_in=20; ap.max=1; ap.sub=1; // result = 20, error = 0
```

6.8 Data Packing & Permutation

6.8.1 PACK (ap.pack)

Concatenates the lower 16 bits of b_in and a_in into a 32-bit result: {b_in[15:0], a_in[15:0]}.

Operation Control Signals

- Primary Enable: ap.pack = 1

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0

Behavior Specification

Condition	Result	Error
Valid PACK	result = {b_in[15:0], a_in[15:0]}	0

Invalid — other ap.* field ≠ 0 and csr_ren_in = 1	result = 0	1
---	------------	---

Worked Example

```
a_in=0x0000_1234; b_in=0x0000_5678; ap.pack=1; // result = 0x5678_1234, error = 0
```

6.8.2 GREV — Byte-Reverse subset (ap.grev)

This implementation supports one encoding of the generalized-reverse family: full byte reversal, selected when b_in[4:0] = 24 (5'b11000). Other b_in[4:0] encodings are not implemented as distinct permutations by this block.

Operation Control Signals

- Primary Enable: ap.grev = 1
- Required Encoding: b_in[4:0] = 24

Execution Guard Conditions

- All other ap.* fields must be 0
- csr_ren_in must be 0
- b_in[4:0] must equal 24 (see Section 7.1 for the undefined-encoding assumption)

Behavior Specification

Condition	Result	Error
Valid, b_in[4:0] = 24	result = byte-reversed a_in	0
b_in[4:0] ≠ 24 (undefined encoding)	result = 0	1 — ADOPTED ASSUMPTION, see 7.1
Invalid — other ap.* field ≠ 0 and csr_ren_in = 1	result = 0	1

Worked Example

```
a_in=0x1234_5678; b_in=24; ap.grev=1; // result = 0x7856_3412, error = 0
```

6.9 CSR Operations

6.9.1 CSR Bypass Read (csr_ren_in, ap = 0)

When csr_ren_in = 1 and no bit-manipulation control field is set, the BMU passes csr_rddata_in through as the result. This is a distinct valid mode, separate from every bit-manipulation operation above.

Operation Control Signals

- Enable: csr_ren_in = 1

Execution Guard Conditions

- All ap.* fields must be 0

Behavior Specification

Condition	Result	Error
Valid CSR bypass read	result = csr_rddata_in	0

Worked Example

```
csr_ren_in=1; ap=0; csr_rddata_in=0xABCD_1234; // result = 0xABCD_1234, error = 0
```

6.9.2 CSR Write Data (ap.csr_write)

Selects the data to be written to a CSR: b_in for an immediate-form write, a_in otherwise.

Operation Control Signals

- Primary Enable: ap.csr_write = 1
- Mode Select: ap.csr_imm

Execution Guard Conditions

- csr_ren_in should be 0 for a pure write (see Section 5 for combined-request behavior)

Behavior Specification

Condition	Result	Error
ap.csr_imm = 1	result = b_in	0
ap.csr_imm = 0	result = a_in	0

7. Adopted Assumptions Pending Design-Team Confirmation

Note: The two items in this section were NOT explicitly answered by the spec owner at the time of writing. Each adopts the interpretation judged safest for a datapath block of this kind, and is clearly separated here so it can be revised without a full spec rewrite if the design team's answer differs.

7.1 GREV undefined-encoding behavior

Adopted: b_in[4:0] ≠ 24 while ap.grev = 1 is treated as an invalid/undefined encoding → result = 0, error = 1.

Rationale: the block implements only one encoding of the generalized-reverse permutation family (full byte-reverse). For every other operation in this specification, an input combination outside the operation's defined encoding space is treated as an error rather than silently returning a plausible-looking 0 — this keeps behavior consistent across the whole unit and avoids masking a decode/software bug behind a silently-wrong-but-plausible result. Silently returning 0 without an error flag would be the one inconsistent case in the entire specification.

7.2 CSR-conflict condition scope (AND vs. OR)

Adopted: within a given operation's guard conditions, the invalid/error condition is: (that operation's own primary enable is asserted) AND ((another ap.* field is asserted) OR (csr_ren_in = 1)). It is NOT a global rule that csr_ren_in = 1 is invalid on its own.

Rationale: Section 6.9.1 defines csr_ren_in = 1 with ap = 0 as a distinct, valid CSR bypass-read mode (result = csr_rddata_in, error = 0). A global rule making csr_ren_in = 1 invalid by itself would directly contradict that mode. Scoping the OR-condition to apply only once an operation's own enable field is asserted preserves both: the bypass-read mode stays valid, and every operation's conflict detection still triggers on either guard violation independently, consistent with each operation's stated guard conditions.

8. Known Documentation Gaps (Out of Current Scope)

Note: These fields were found to be functionally active in the RTL control struct but have no behavior specification in the original document. They are listed here, not specified, so scope is explicit rather than silently

incomplete. Recommend confirming with the design team whether each is in scope for this verification effort before writing tests against it.

Field(s)	Apparent Function	Status
ap.min, ap.clz, ap.rol	Minimum, count-leading-zeros, rotate-left — natural counterparts to documented MAX / CTZ / ROR.	No behavior table in spec. Needs a written specification before test development.
ap.bset, ap.bclr	Single-bit set / clear — counterparts to documented BINV.	No behavior table in spec.
ap.packu, ap.packh, ap.gorc	Additional pack variants and OR-compression (ORC.B), referenced only in a one-line feature summary elsewhere in the original document.	No behavior table, no worked example.
ap.beq, ap.bne, ap.blr, ap.bge, ap.jal, ap.predict_t, ap.predict_nt	Branch-resolution / prediction signals observed in the same control struct and module.	Appear unrelated to bit-manipulation. Recommend confirming whether this module's verification scope extends beyond the documented BMU functionality.

Appendix A: Revision Summary vs. v1.1

- Corrected error=0 → error=1 on 'other fields ≠ 0' invalid rows for OR, XOR, SRL, SRA, ROR, BINV.
- Corrected CTZ(0) result from 0 to 32.
- Added Section 3 (Timing, Reset, Test-Mode) — previously undocumented.
- Added explicit guard-condition philosophy (Section 4) unifying language used inconsistently across the original per-operation sections.
- Added Section 7 documenting two assumptions adopted where the original spec was ambiguous and unconfirmed.
- Added Section 8 cataloguing control-struct fields with no specification, found during RTL cross-check.